FINAL EVOLUTION LAB — MASTER ENGINE BRIEF

Seele AI Compilation Target · Production Specification v1.0

> **Classification:** 3D Neuro-Athletic Performance Environment
> **Engine:** Unreal Engine 5.7 (native iOS Shipping + Linux E3DS)
> **Shell:** Native iOS Swift app with embedded WKWebView dashboard overlay
> **Backend:** FastAPI (MongoDB + Firestore + Firebase Data Connect / PostgreSQL)
> **Branch:** `anti-gravity-fel` · Commit `9519541`
> **Date:** 2026-05-22

---

§1 SYSTEM ARCHITECTURE SUMMARY

Final Evolution Lab is a **3D Neuro-Athletic Performance Environment** built natively inside Unreal Engine 5.7 with an integrated native iOS shell and cloud data backend. The system fuses real-time biometric telemetry from Apple HealthKit into deterministic arcade physics, dynamic difficulty adjustment, and a three-tier neurocognitive resiliency engine that models prefrontal cortex adaptation under high-stimulus gameplay.

1.1 Runtime Layer Stack

(* = new components defined in this brief)

1.2 Source Paths

Layer	Path	Stack
UE C++ Subsystems	`UnrealIntegration/Source/FinalEvolutionLab/`	C++/ObjC++, UE 5.7
UE Blueprints + Config	`UnrealStarter/BasketballGame/`	UE 5.7 BPs, ArenaSettings.json
Swift App	`FinalEvolutionLab/`	Swift, SwiftUI, Firebase, HealthKit
React Dashboard	`frontend/src/`	React 19, Tailwind, Axios
Backend	`backend/`	FastAPI, MongoDB, PayPal REST
Infra	`infra/`	Pulumi, UE5 config, CI
Data Connect	`dataconnect/`	GraphQL, PostgreSQL

1.3 Cooked Map Registry (Source of Truth: DefaultGame.ini [EmergentPlayMap])

Map Token	Cooked Path	Modes Hosted	In MapsToCook
VeniceBeach	`/Game/FEL/Venues/VeniceBeach/VeniceBeach`	basketball_h2h, basketball_dunk, basketball_3v3, surfing, court_carnival	✅
Dojo	`/Game/FEL/Venues/Dojo/Dojo`	karate_h2h, karate_endless	✅
BaseballPark	`/Game/FEL/Venues/BaseballPark/BaseballPark`	baseball	✅
Gridiron	`/Game/FEL/Venues/Gridiron/Gridiron`	football	✅
SoccerStadium	`/Game/FEL/Venues/SoccerStadium/SoccerStadium`	soccer	✅
Links	`/Game/FEL/Venues/Links/Links`	golf	✅
TennisCourt	`/Game/FEL/Venues/TennisCourt/TennisCourt`	tennis	✅
SandCourt	`/Game/FEL/Venues/SandCourt/SandCourt`	volleyball	✅
TrainingFloor	`/Game/FEL/Venues/TrainingFloor/TrainingFloor`	gymnastics	✅
NeuroArena	`/Game/FEL/Venues/NeuroArena/NeuroArena`	brain_brawl, who_scene_it	✅
Luma_Venice_Shop	`/Game/FEL/Venues/Luma_Venice_Shop/Luma_Venice_Shop`	market_browse	✅
Skate_Park	`/Game/FEL/Venues/Skate_Park/Skate_Park`	skateboarding	❌ staging
Mountain_Slope		snowboarding	❌ staging

Map Token	Cooked Path	Modes Hosted	In MapsToCook
	`/Game/FEL/Venues/Mountain_Slope/Mountain_Slope`		

§2 COMPLETE GAME MODE REGISTRY (19 Modes)

2.1 Production Modes (12)

#	mode_id	Display	Venue	Input	Sport	MP	PRQ Wt	Target	Time
1	basketball_h2h	Street · 1v1	Venice Beach	charge	Basketball	real-time	1.2	3	∞
2	basketball_dunk	Dunk Contest	Venice Beach	charge	Basketball	real-time	1.0	21	∞
3	basketball_3v3	Street · 3v3	Venice Beach	charge	Basketball	real-time	1.3	11	∞
4	karate_h2h	Karate · 1v1	Dojo	charge	Combat	real-time	1.4	5	150s
5	karate_endless	Karate · Endless	Dojo	charge	Combat	solo	1.4	∞	∞
6	baseball	Home Run Derby	BaseballPark	swipe	Field	turnBased	1.0	6	300s
7	football	Kick Return	Gridiron	kickReturn	Field	turnBased	1.5	3	240s
8	soccer	Penalty Shootout	SoccerStadium	penaltyKick	Field	real-time	1.1	5	180s
9	golf	Closest to Pin	Links	swipeGolf	Precision	turnBased	0.9	5	300s
10	tennis	Rally Ace	TennisCourt	rallyAce	Precision	real-time	1.1	5	120s
11	volleyball	Rally Ace	SandCourt	rallyAce	Field	real-time	1.2	5	120s
12	surfing	Surf Line	Venice Beach	rhythmTap	Board	real-time	1.05	∞	180s

2.2 Staging Modes (4)

#	mode_id	Display	Venue	Input	Blocker
13	skateboard-ing	Skate Line	Skate_Park	rhythmTap	Map not in MapsToCook; Emergent-PlayMap mis-routed
14	snowboard-ing	Snow Line	Moun-tain_Slope	rhythmTap	Map not in MapsToCook; both Emer-gentPlayMap + ArenaSet-tings wrong
15	gymnastics	Gymnastics	TrainingFloor	rhythmTap	PRQ mode-Weight un-defined
16	brain_brawl	Brain Brawl	NeuroArena	tap	Non-standard session endpoint; no shards/PRQ delta

2.3 Preview Modes (2)

#	mode_id	Display	Venue	Blocker
17	who_scene_it	Who Scene It	NeuroArena	Missing: Swift enum, ue_mode_maps, VenueRegistry, EmergentPlay-Map, session re-ceipt, scoring
18	court_carnival	Court Carnival	VeniceBeach	Rename from mario_party_fev er; all registries incomplete

2.4 Non-Game Module (1)

#	mode_id	Display	Venue	Purpose
19	market_browse	Module Library	Luma_Venice_Shop	3D shop browsing. No scoring, no session, no PRQ.

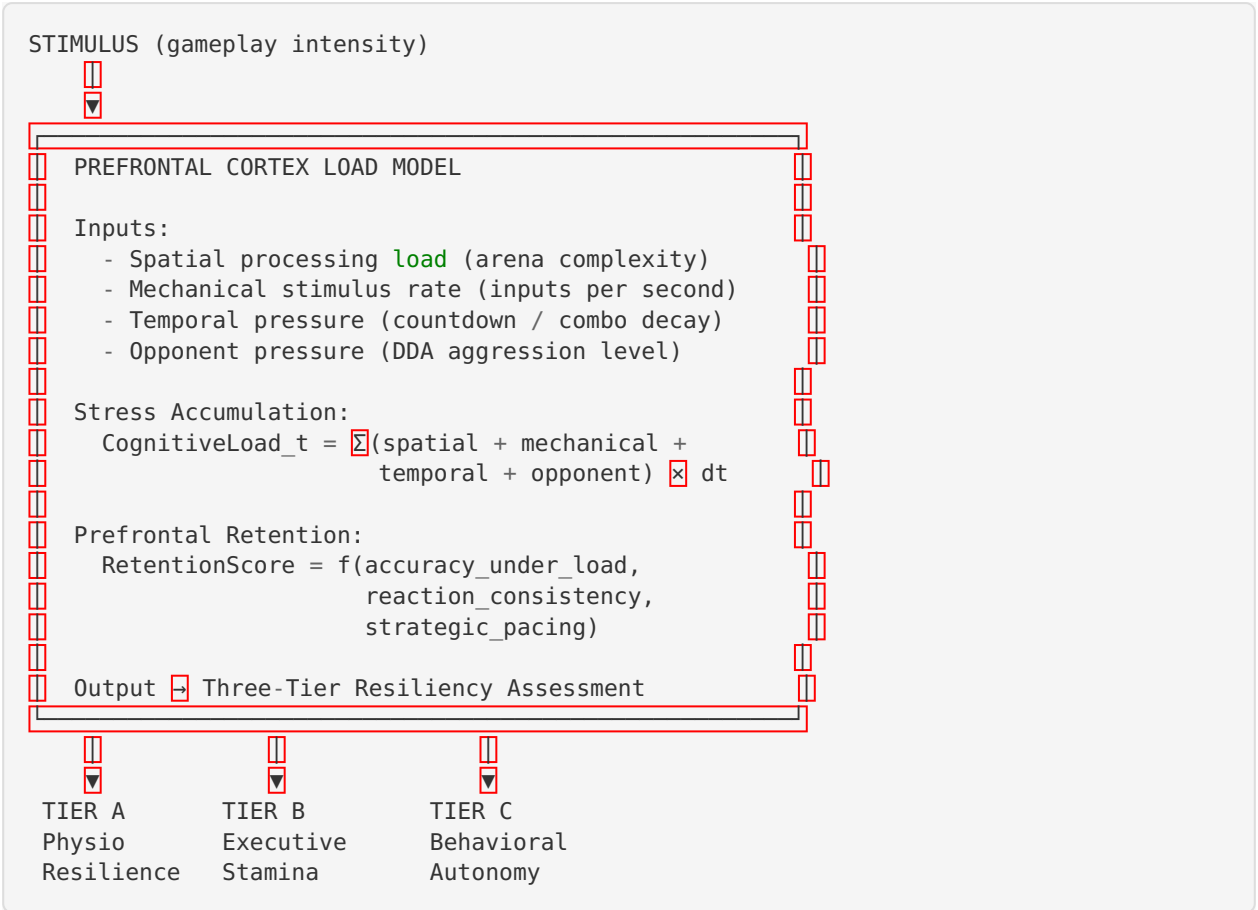
---

§3 MENTAL RESILIENCY & NEUROCOGNITIVE PERFORMANCE ENGINE

3.1 Stress Adaptation Training Loop

The Mental Resiliency Engine translates the neuroscience of prefrontal cortex retention of cognitive control under high spatial/mechanical stimulus into deterministic, trackable game mechanics. The system operates across three measurement tiers that run in parallel during every gameplay session, feeding a composite **Mental Resiliency Index (MRI)** score (0–100).

Neurological Model



Integration With Existing PRQ Pipeline

The MRI engine extends (does not replace) the existing PRQ scoring chain:

```
HealthKit → PRQ Score (0–100) → Neural Readiness Grade
                                          │
                                          ▼
                                ArcadePhysics.fromPRQ()
                                          │
                                          ├── hangTimeMultiplier
                                          ├── explosiveFirstStep
                                          ├── comboDecayRate
                                          ├── criticalHitChance
                                          ├── neuralBurstActive
                                          └── auraLevel
                                          │
                                [NEW] MentalResiliencyEngine
                                          │
                                          ├── Tier A: autonomicRecoveryVelocity
                                          ├── Tier B: executiveStaminaIndex
                                          ├── Tier C: pacingEfficiencyScore
                                          └── Composite: MRI (0–100)
                                                    │
                                                    ▼
                                MRI feeds back into:
                                  - DDA scaledAggression()
                                  - GoldenEraComboEngine bonuses
                                  - Session receipt PRQ delta
                                  - Avatar performance attributes
```

3.2 TIER A — Physiological Resilience: Autonomic Recovery Velocity (ARV)

Definition

Autonomic Recovery Velocity measures the time delta (in seconds) required for a user's Heart Rate Variability (HRV SDNN) to return to a pre-session baseline state immediately following high-intensity gameplay sequences. This maps the parasympathetic nervous system's capacity to restore homeostasis after sympathetic arousal — a direct biomarker of stress resilience.

Data Source

```
HealthKit HRV SDNN samples (ms)
  → HealthKitService.fetchLatestQuantity(type: .heartRateVariabilitySDNN)
  → Weekly baseline: HealthKitService.fetchWeeklyHRVAverage() (7-day mean)
  → Intra-session sampling: 30s polling interval during active gameplay
```

ARV Equation

```
// Inputs
HRV_baseline = weeklyHRVAverage (ms)          // 7-day rolling mean from HealthKit
HRV_stress   = min(HRV samples during peak intensity window) (ms)
HRV_current  = latest HRV sample post-intensity (ms)
t_peak_end   = timestamp when peak intensity window closes (epoch ms)
t_recovery   = timestamp when HRV_current >= HRV_baseline * RecoveryThreshold (epoch m
s)

// Recovery threshold
RecoveryThreshold = 0.90  // 90% of baseline = "recovered"

// Core calculation
HRV_depression = max(0, HRV_baseline - HRV_stress)        // ms drop during stress
Recovery_delta = (t_recovery - t_peak_end) / 1000.0        // seconds to recover

// ARV Score (0–100, higher = faster recovery = more resilient)
if HRV_depression < 5.0:
    ARV = 95.0  // minimal depression = elite autonomic control
elif Recovery_delta <= 0:
    ARV = 90.0  // immediate recovery
else:
    raw_ARV = 100.0 - (Recovery_delta / MaxRecoveryWindow * 100.0)
    ARV = clamp(raw_ARV, 0, 100)

// MaxRecoveryWindow = 180 seconds (3 minutes)
// Faster recovery → higher ARV → higher MRI contribution
```

Variable Handles

```
@variable ARV_HrvBaseline: Float       // weeklyHRVAverage in ms
@variable ARV_HrvStress: Float         // minimum HRV during intensity peak
@variable ARV_HrvCurrent: Float        // latest HRV post-intensity
@variable ARV_RecoveryDeltaSec: Float  // seconds to reach 90% baseline
@variable ARV_Score: Float             // 0–100 normalized
@variable ARV_MaxRecoveryWindow: Float = 180.0
@variable ARV_RecoveryThreshold: Float = 0.90
```

Peak Intensity Detection

A “high-intensity gameplay sequence” is detected when ANY of:

- `DDA.aggression >= 1.2` for ≥ 10 consecutive seconds
- `InputManager.inputsPerSecond >= 4.0` sustained for ≥ 8 seconds
- `GoldenEraComboEngine.chainLength >= 4` (high-complexity combo execution)
- `ArcadePhysics.neuralBurstActive == true` (neural drive ≥ 80)

Gameplay Impact

ARV Score	Grade	Arcade Modifier	DDA Effect
≥ 85	RAPID	comboDecayRate × 0.85 (slower decay)	AI aggression ceiling −0.1
65–84	ADAPTIVE	comboDecayRate × 0.95	No change
40–64	STANDARD	comboDecayRate × 1.00	No change
< 40	DELAYED	comboDecayRate × 1.15 (faster decay)	Prompt recovery state

---

3.3 TIER B — Executive Stamina Index (ESI): Cognitive Deceleration Curve

Definition

The Executive Stamina Index tracks passive behavioral telemetry to measure cognitive fatigability over a sustained training session. It monitors three signals that indicate prefrontal cortex fatigue without requiring explicit user reporting:

1. **Context-Switching Friction (CSF):** Reaction time degradation when transitioning between action types (e.g., offense → defense, shoot → dodge)
2. **Error-Escalation Frequency (EEF):** Rate at which consecutive errors increase after the first error in a sequence
3. **Input Lag Drift (ILD):** Progressive increase in average input latency compared to session-start baseline

Data Sources (Passive Behavioral Telemetry)

```
// Context-Switching Friction
InputManager.lastActionType    // sprint, shoot, block, dodge, etc.
InputManager.actionTimestamp   // when action was registered
AvatarStateMachine.currentState // idle, sprint, dunk, block, counter, etc.
Transition_latency = time between state exit and next valid input

// Error-Escalation Frequency
GoldenEraComboEngine.apexGrade  // PERFECT, GREAT, GOOD, OK, MISS
ArcadePhysics.lastShotResult    // hit, miss, blocked, critical
Sequential error tracking per 60-second windows

// Input Lag Drift
InputManager.inputBuffer        // 4-frame window (0.067s)
Average input registration latency vs session-start baseline
Rolling 30-second windows
```

ESI Equations

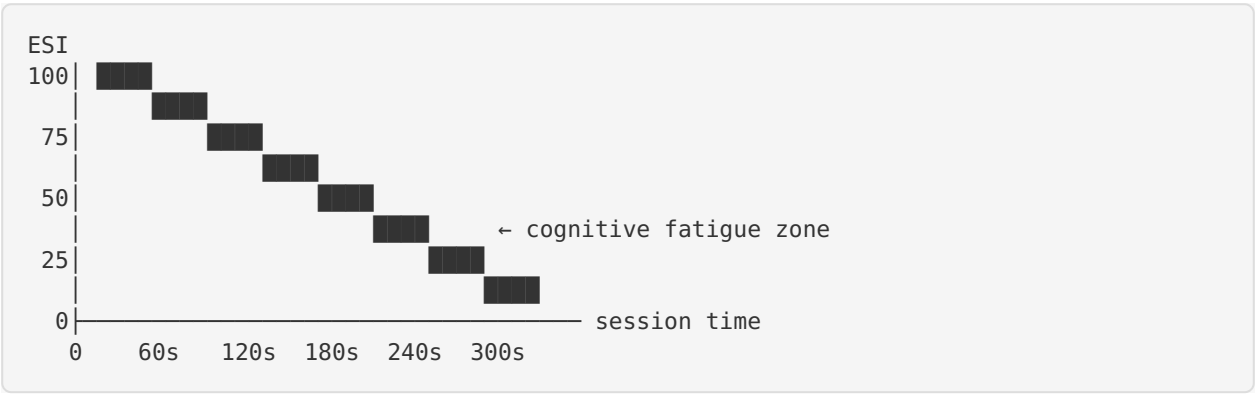
```
// 1. Context-Switching Friction (CSF)
//    Measures reaction time increase when switching between action contexts
CSF_baseline = mean(transition_latency[0:first_60s])  // first minute baseline
CSF_current  = mean(transition_latency[last_30s])      // rolling 30s window
CSF_ratio    = CSF_current / max(CSF_baseline, 0.001)
CSF_score    = clamp(100.0 - (CSF_ratio - 1.0) * 200.0, 0, 100)
// CSF_ratio of 1.0 = no degradation (100). Ratio 1.5 = 50% slower (0).

// 2. Error-Escalation Frequency (EEF)
//    Tracks whether errors compound (each error makes next more likely)
errors_per_window = count(MISS or blocked or fail) per 60s window
windows = sliding 60s windows across session
if len(windows) >= 3:
    error_slope = linear_regression_slope(windows[-3:])
    EEF_score = clamp(100.0 - error_slope * 500.0, 0, 100)
    // Positive slope (escalating errors) → lower score
    // Negative slope (improving) → higher score
else:
    EEF_score = 75.0  // insufficient data, neutral

// 3. Input Lag Drift (ILD)
//    Detects progressive input latency increase indicating cognitive slowing
ILD_baseline = mean(input_registration_latency[0:first_60s])
ILD_current  = mean(input_registration_latency[last_30s])
ILD_delta_ms = ILD_current - ILD_baseline
ILD_score    = clamp(100.0 - ILD_delta_ms * 5.0, 0, 100)
// Each 1ms of drift = -5 points. 20ms drift = score 0.

// COMPOSITE Executive Stamina Index
ESI = (CSF_score * 0.35) + (EEF_score * 0.40) + (ILD_score * 0.25)
```

Variable Handles

```
@variable ESI_CSF_Baseline: Float       // first-minute transition latency mean
@variable ESI_CSF_Current: Float        // rolling 30s transition latency mean
@variable ESI_CSF_Score: Float          // 0–100
@variable ESI_EEF_ErrorSlope: Float     // errors per window linear slope
@variable ESI_EEF_Score: Float          // 0–100
@variable ESI_ILD_Baseline: Float       // first-minute input latency mean (ms)
@variable ESI_ILD_Current: Float        // rolling 30s input latency mean (ms)
@variable ESI_ILD_DriftMs: Float        // delta from baseline
@variable ESI_ILD_Score: Float          // 0–100
@variable ESI_Composite: Float          // 0–100 weighted composite
```

Cognitive Deceleration Curve

The ESI composite plotted over session time produces the **Cognitive Deceleration Curve** — a per-session graph showing executive function degradation:



Gameplay Impact

ESI Score	Grade	Effect
≥ 80	SHARP	Perfect guard window +10% (InputManager.perfectGuardWindow × 1.1)
60–79	FOCUSED	No modification
40–59	FATIGUING	QTE apex window widens +15% (GoldenEraComboEngine.apexWindow × 1.15) — compensatory assist
< 40	DEPLETED	Prompt tactical pause; reduce DDA aggression floor by 0.1; trigger Tier C pacing evaluation

---

3.4 TIER C — Behavioral Autonomy & Self-Regulation: Pacing Efficiency

Definition

Pacing Efficiency monitors whether a user proactively triggers low-intensity recovery states or structured tactical pauses when flagged with high cognitive strain. This measures the executive function of **self-regulation** — the ability to recognize internal fatigue signals and act on them rather than persisting to failure.

Detection Logic

```
// Pacing events — user-initiated recovery actions
PacingEvent triggers when:
  1. User voluntarily pauses (menu/pause input) during active gameplay
  2. User selects recovery mode when PRQ < 40 (grade RECOVERING)
  3. User reduces input rate by >50% for ≥5s during high-DDA period
  4. User transitions to low-intensity activity after high-intensity sequence
     (AvatarStateMachine: sprint/dunk/special → idle/block for ≥3s)

// Strain context — was a pacing event "smart"?
StrainFlagged = true when ANY of:
  - ESI_Composite < 50 (cognitive fatigue)
  - ARV_Score < 50 (slow autonomic recovery)
  - neuralReadinessGrade == RECOVERING
  - DDA.aggression >= 1.3 (high AI pressure)
  - errorEscalation slope > 0.05 (errors compounding)

// Pacing Efficiency Score
smartPauses     = count(PacingEvents where StrainFlagged == true)
missedPauses    = count(windows where StrainFlagged and no PacingEvent within 30s)
forcedRecoveries = count(health_depleted or timeout events)

PacingEfficiency = clamp(
    70.0
    + smartPauses * 8.0          // reward proactive pacing
    - missedPauses * 12.0        // penalize pushing through strain
    - forcedRecoveries * 20.0    // heavily penalize forced stops
    , 0, 100
)
```

Variable Handles

```
@variable PACE_SmartPauses: Int          // pacing events during strain
@variable PACE_MissedPauses: Int         // strain windows without pacing
@variable PACE_ForcedRecoveries: Int     // involuntary stops
@variable PACE_EfficiencyScore: Float    // 0–100
@variable PACE_StrainFlagged: Bool       // current strain state
@variable PACE_LastPauseTimestamp: Float  // epoch ms of last pacing event
```

Gameplay Impact

Pacing Score	Grade	Effect
≥ 75	STRATEGIC	+5% shard bonus on session completion (“Self-Regulation Bonus”)
50–74	REACTIVE	No modification
25–49	RECKLESS	UI hint: “Recovery recommended” overlay pulse
< 25	OVERDRIVEN	Mandatory 15s cooldown prompt; DDA floor drops to 0.6

3.5 Composite Mental Resiliency Index (MRI)

Formula

```
MRI = (ARV_Score * 0.30) + (ESI_Composite * 0.45) + (PACE_EfficiencyScore * 0.25)

// Weights rationale:
//   ESI (0.45): Executive stamina is the strongest predictor of
//               sustained performance under load
//   ARV (0.30): Physiological recovery directly maps to stress resilience
//   PACE (0.25): Behavioral self-regulation is trainable and
//                rewards metacognition
```

MRI Grade Thresholds

MRI Score	Grade	HUD Indicator	Avatar Effect
≥ 85	UNBREAKABLE	🟣 Purple pulse aura	auraLevel = MAX_INTENT; +0.15 to neuralFocus attribute
65–84	RESILIENT	🔵 Cyan steady glow	auraLevel = PRIMED; +0.08 to neuralFocus
45–64	ADAPTING	🟢 Green ambient	auraLevel = ACTIVE; no modifier
< 45	VULNERABLE	🟡 Yellow flicker	auraLevel = BASELINE; prompt recovery sequence

MRI → PRQ Session Delta Modifier

```
prqDelta = PRQ.modeReward(mode, won, tied, combo, criticals, scoreDiff)
mriBonus = (MRI / 100.0) * 0.5   // 0–0.5 additional PRQ points
adjustedDelta = prqDelta + mriBonus

// Example: Win with MRI 90 → prqDelta 2.4 + mriBonus 0.45 = 2.85
// Example: Loss with MRI 30 → prqDelta 0.2 + mriBonus 0.15 = 0.35
```

Session Receipt Extension


```
{
  "session_id": "string",
  "mode_id": "string",
  "score": 0,
  "duration_seconds": 0,
  "xp_awarded": 0,
  "shards_awarded": 0,
  "prq_delta": 0.0,
  "neurocognitive": {
    "mri_score": 0.0,
    "mri_grade": "RESILIENT",
    "tier_a_arv": 0.0,
    "tier_b_esi": 0.0,
    "tier_b_csf": 0.0,
    "tier_b_eef": 0.0,
    "tier_b_ild": 0.0,
    "tier_c_pacing": 0.0,
    "smart_pauses": 0,
    "missed_pauses": 0,
    "forced_recoveries": 0,
    "cognitive_deceleration_curve": [[0, 100], [60, 95], [120, 82], [180, 68]]
  }
}
```


Firestore Schema Extension

```
users/{uid}/neurocognitive_sessions/{sessionId}
  ├── mri_score: number (0–100)
  ├── mri_grade: string
  ├── tier_a: { arv_score, hrv_baseline, hrv_stress, recovery_delta_sec }
  ├── tier_b: { esi_composite, csf_score, eef_score, ild_score, decel_curve: array }
  ├── tier_c: { pacing_score, smart_pauses, missed_pauses, forced_recoveries }
  ├── mode_id: string
  ├── session_duration_sec: number
  ├── created_at: Timestamp
```

UE Bridge Payload Extension

Add to existing `UnrealSystemScanPayload`:

```
{
  "neurocognitive": {
    "schemaVersion": 1,
    "mriScore": 72.0,
    "mriGrade": "RESILIENT",
    "tierA_ARV": 78.0,
    "tierB_ESI": 65.0,
    "tierC_Pacing": 80.0,
    "strainFlagged": false,
    "comboDecayModifier": 0.95,
    "perfectGuardModifier": 1.0,
    "qteWindowModifier": 1.0,
    "ddaFloorOverride": -1.0,
    "pacingSuggestion": "none"
  }
}
```


---

§4 SEELE AI PARSING CONFIGURATIONS

4.1 Player State Handles

```
// Core movement states (from AvatarStateMachine — 22 states)
@state Idle                // default standing pose
@state Sprinting           // full-speed locomotion
@state Gathering           // charge/gather animation (pre-shot, pre-dunk)
@state Dribbling           // basketball dribble locomotion
@state Shooting            // shot release animation
@state Dunking             // airborne dunk sequence
@state AirTime             // generic airborne (jump apex)
@state Landing             // ground contact recovery
@state Blocking            // defensive guard pose
@state Countering          // parry/counter-attack
@state Vanishing           // evasion dash (scale 0.75x)
@state HitStun             // stagger from impact
@state Special             // signature move (scale 1.14x)
@state Swinging            // bat/club/racket swing
@state Kicking             // soccer/football kick
@state Spiking             // volleyball spike
@state Serving             // tennis/volleyball serve
@state CatchingBall        // ball reception
@state Celebrating         // post-score celebration

// Neurocognitive states (NEW)
@state StressRecoveryState // Tier C pacing pause — low-intensity recovery mode
@state CognitiveCooldown   // ESI < 40 mandatory 15s cooldown
@state NeuralBurst         // neuralDrive ≥ 80, active burst mode

// PRQ-derived states
@state RecoveryMode        // PRQ grade RECOVERING (< 40)
@state PrimedMode          // PRQ grade PRIMED (60–79)
@state EliteMode           // PRQ grade ELITE (≥ 80)
```

4.2 Performance Variable Handles

```
// Existing (from PRQScoring, ArcadePhysics, DynamicDifficulty)
@variable PRQ_Score: Float              // 0–100, default 75
@variable PRQ_Normalized: Float         // PRQ / 100, clamped [0,1]
@variable NeuralDrive: Float            // 0–100
@variable NeuralReadinessGrade: Enum    // ELITE | PRIMED | READY | RECOVERING
@variable HangTimeMultiplier: Float     // 1.0 + norm*1.8 + neural*0.4
@variable ExplosiveFirstStep: Float     // 0.3 + norm*0.7
@variable ComboDecayRate: Float         // 5.0 - norm*3.0  (2.0–5.0)
@variable MaxComboMultiplier: Float     // 2.0 + norm*3.0  (2.0–5.0)
@variable CriticalHitChance: Float      // 0.05 + norm*0.2 + neural*0.1
@variable NeuralBurstActive: Bool       // neuralDrive >= 80
@variable NeuralBurstMultiplier: Float  // 1.5 when active
@variable AuraLevel: Enum               // BASELINE | ACTIVE | PRIMED | MAX_INTENT
@variable SpeedMultiplier: Float        // 0.9–1.15 (grade-based)
@variable HangTimeBonus: Float          // -0.10–+0.30 (grade-based)
@variable DDA_Aggression: Float         // 0.6–1.4
@variable DDA_DifficultyTier: Enum      // ROOKIE | DEVELOPING | ADVANCED | ELITE | LE
GENDARY

// Biomechanics (from BiomechanicsAudit)
@variable Biomech_AnkleScore: Float     // 0–100
@variable Biomech_KneeScore: Float      // 0–100
@variable Biomech_HipScore: Float       // 0–100
@variable Biomech_Grade: Enum           // ELITE | PRIMED | DEVELOPING | FOUNDATION
@variable Biomech_IsPrimed: Bool        // grade >= PRIMED

// Combo Engine (from GoldenEraComboEngine)
@variable Combo_ChainLength: Int        // 0–6 max
@variable Combo_Multiplier: Float       // 1.0–4.0
@variable Combo_ChainWindow: Float      // 0.5s
@variable Combo_StyleLandingWindow: Float // 0.35s
@variable QTE_ApexWindow: Float         // 0.4s
@variable QTE_Grade: Enum               // PERFECT | GREAT | GOOD | OK | MISS
@variable QTE_GradeMultiplier: Float    // 2.0 | 1.5 | 1.2 | 1.0 | 0.3

// Input (from InputManager)
@variable Input_BufferWindow: Float     // 0.067s (4 frames)
@variable Input_ComboChainWindow: Float // 0.4s
@variable Input_DoubleTapWindow: Float  // 0.25s
@variable Input_PerfectGuardWindow: Float // mode-specific
@variable Input_FaceButton: Enum        // SQUARE | TRIANGLE | CIRCLE | CROSS
@variable Input_DunkModifier: Enum      // STANDARD | FLASHY | POWER | SIGNATURE

// Neurocognitive (NEW)
@variable MRI_Score: Float              // 0–100 composite
@variable MRI_Grade: Enum               // UNBREAKABLE | RESILIENT | ADAPTING | VUL-
NERABLE
@variable TierA_ARV: Float              // 0–100
@variable TierB_ESI: Float              // 0–100
@variable TierB_CSF: Float              // 0–100 context-switching friction
@variable TierB_EEF: Float              // 0–100 error-escalation frequency
@variable TierB_ILD: Float              // 0–100 input lag drift
@variable TierC_Pacing: Float           // 0–100
@variable StrainFlagged: Bool           // active cognitive strain
```

4.3 HUD Specifications

Scoreboard Panel (Top-Center)

```
┌──────────────────────────────────────────────────┐
│  PLAYER: 7          MODE: Street · 1v1     AI: 5 │
│  ──────────────────────────────────────────────  │
│  TIME: 2:34        VENUE: Venice Beach Court     │
└──────────────────────────────────────────────────┘

Position: top-center, 10% from top edge
Width: 60% viewport
Background: #0F0F13 at 85% opacity
Font: Barlow Condensed Bold, 28pt (scores), 14pt (labels)
Color: #00E5FF (cyan) primary, #FFFFFF scores
```

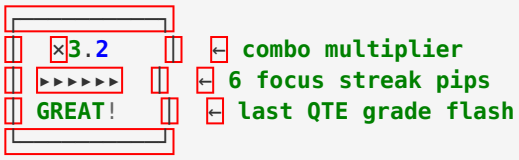
Active Statistical Ticker (Top-Right)

```
┌──────────────────────────┐
│  PRQ  78  ▲ PRIMED       │
│  MRI  72  ● RESILIENT    │
│  ARV  85  ⚡ RAPID       │
│  ESI  65  ○ FOCUSED      │
│  XP   +42                │
│  ◆    +50 shards         │
└──────────────────────────┘

Position: top-right, 5% from edge
Width: 20% viewport
Update frequency: 1Hz (stats), real-time (XP/shards on award)
Background: #16161A at 90% opacity
Font: IBM Plex Sans, 12pt
Accent: per-metric color (PRQ=#00E5FF, MRI=#A855F7, ARV=#22C55E, ESI=#F59E0B)
```

Takeover Meter (Left Edge, Vertical)

```
│▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓│ 100% ← MAX INTENT (purple)
│▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓│  80% ← PRIMED (cyan)
│▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓│  60% ← ACTIVE (green)
│▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓│  40% ← BASELINE (dim)
│▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓│   0%

Position: left edge, vertically centered, 3% from edge
Height: 40% viewport
Width: 24px
Fill: gradient from current auraLevel color
Glow: 4px bloom at current fill level
Maps to: NeuralDrive (0–100)
Updates: every frame (smooth lerp 0.1s)
```

Combo Multiplier / Focus Streak Indicator (Center-Right, Floating)

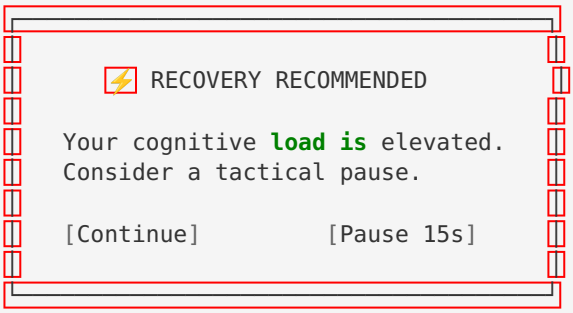



```
Position: center-right, floating at 70% X, 40% Y
Size: dynamic (scales 1.0→1.3x on multiplier increase)
Animation: punch-scale on increment, shake on MISS, gold burst on PERFECT
Font: JetBrains Mono Bold, 36pt (multiplier), 14pt (grade)
Color: #00FF9D (multiplier), grade-specific color
  PERFECT: #FFD700 (gold)
  GREAT:   #00E5FF (cyan)
  GOOD:    #00FF9D (green)
  OK:      #FFFFFF (white)
  MISS:    #FF3366 (red)
Decay: multiplier text fades if comboDecayRate timer expires
Focus streak pips: one pip per consecutive successful input (max 6)
  Each pip is a 6px circle, filled = active, dimmed = expired
  Fills left-to-right; all pips filled triggers Neural Burst visual
```

Neurocognitive Overlay (Bottom-Left, Subtle)


```
Position: bottom-left, 5% from edges
Width: 25% viewport
Visibility: fades in only during active session, auto-hides in menus
Background: #050505 at 80% opacity
Bar: gradient fill (#A855F7 → #00E5FF)
Sub-indicators (CSF/EEF/ILD): 8px dots, green (≥60), yellow (40–59), red (<40)
Pacing label: updates on pacing event detection
Font: IBM Plex Sans, 11pt
```

Recovery Prompt (Center Overlay, Conditional)



Trigger: ESI_Composite < 40 **OR** PACE_EfficiencyScore < 25
Position: viewport center
Background: #0F0F13 **at** 95% opacity **with** 2px #FF3366 border
Font: Barlow Condensed 24pt (header), IBM Plex Sans 14pt (body)
Buttons: #16161A background, #00E5FF **text**
Auto-dismiss: **none** (requires **user input**)
Frequency: **max** once per 120s

4.4 3D Environmental Manifest

Primary Venue: Venice Beach Court (VeniceBeach)

```
@environment VeniceBeach_StreetCourt
  Type: High-fidelity urban outdoor basketball court
  Time of Day: Dusk (golden hour → floodlight transition)
  Lighting:
    - Sun: directional, 15° above horizon, warm amber (#FFB347), intensity 3.2
    - Sky: HDRI volumetric, gradient from deep orange (#FF6B35) to dark blue (#1A1A3E)
    - Floodlights: 4× point lights on 12m poles, cool white (#E0F0FF), intensity 8.0,
      cone angle 45°, volumetric scattering enabled, dynamic shadows
    - Ambient: bounce GI from court surface, warm undertone
    - Rim: backlit cyan accent (#00E5FF) for player silhouette readability

  Court Surface:
    - Material: Worn concrete with painted half-court lines
    - Size: 14m × 15m regulation half-court
    - Texture: PBR — base (grey concrete), roughness map (worn center, fresh paint lin
es),
      normal map (crack patterns), metallic (0.0)
    - Collision: box collider matching painted boundaries, invisible wall at 16m

  Backdrop:
    - Venice Beach boardwalk geometry (low-poly LOD beyond 30m)
    - Palm tree silhouettes (billboard sprites at distance)
    - Chain-link fence boundary (transparent collision mesh)
    - Graffiti wall (south side, emissive details)
    - Crowd: 20 spectator NPC positions (animated idle/cheer cycles)

  Hoop Assembly:
    - Regulation height: 3.05m
    - Backboard: 1.83m × 1.22m, tempered glass material (clear with white rectangle)
    - Rim: 0.457m diameter, orange steel, physics-enabled for ball bounce
    - Net: cloth simulation, 12 chain segments

  Collision Boundaries:
    - Court bounds: 14m × 15m box
    - Out-of-bounds: 2m buffer zone with invisible reset trigger
    - Hoop: sphere collider (score detection), box collider (backboard), cylinder (rim
)
    - Fence: line trace collision at boundary

  Performance Targets:
    - iOS: 60fps at 1920×1080, dynamic resolution 75%–100%
    - Draw calls: ≤ 800 (court + characters + FX)
    - Texture budget: 512MB VRAM
    - Character LOD: 3 levels (hero 15k tris, mid 8k, far 3k)
```

Character Models

```
@character PlayerAvatar
  Rig: UE5 Mannequin-compatible skeleton (67 bones)
  Mesh: Modular (head, torso, arms, legs, feet — separate meshes)
  Polygon count: 12,000–15,000 triangles (hero LOD)
  Textures:
    - Base color: 2048×2048, PBR metallic workflow
    - Normal: 2048×2048 (muscle definition, clothing folds)
    - Roughness: 1024×1024
    - Emissive: 1024×1024 (aura glow layer, intensity driven by AuraLevel)
  Skin Configuration (from AvatarSkinConfig.fromScan):
    PRQ ≥ 80: elitePurple outfit, aura (0.6, 0.2, 1.0)
    PRQ 65–79: cyan outfit, aura (0, 0.95, 0.9)
    PRQ 50–64: blue outfit, aura (0, 0.83, 1.0)
    PRQ < 50: green outfit, aura (0.2, 1.0, 0.4)
  Animation Set:
    22 pose states (AvatarStateMachine) with 0.2s blend transitions
    Pose scale modifiers: jump (0.95w, 1.10h), special (1.14×), vanish (0.75×)
  Creator Card Skins:
    MovementSignature overrides: jumpApex (1.0–1.5×), hangTimeFactor (1.0–1.8×),
    firstStepBurst (1.0–1.5×), limbEmission (0–1.0), trailColor per card

@character OpponentAI
  Same rig/mesh as PlayerAvatar
  DDA-driven animation speed: aggressionMultiplier (0.6–1.4) scales anim playback rate
  Difficulty tier visual: ROOKIE (green trim), DEVELOPING (blue), ADVANCED (cyan),
    ELITE (purple), LEGENDARY (gold + particle trail)
```

Export Configuration

```
@export UE5_Package
  Engine: Unreal Engine 5.7
  Target: iOS Shipping + Linux E3DS
  Rendering: MetalRHI (iOS), Vulkan (Linux)
  Nanite: Disabled (iOS target)
  Lumen: Disabled (use baked + dynamic point lights)
  Virtual Shadow Maps: Enabled with cascade limit 3
  Texture Streaming: Pool 512MB
  LOD: Automatic with 3 tiers (10m, 25m, 50m thresholds)
  Physics: Chaos (default), cloth simulation for net
  Audio: MetaSounds, spatial audio for crowd/ball bounce
  Pak file cooking: per-venue .pak chunks for streaming

@export_compatibility
  Primary: UE 5.7 .uproject (native compilation target)
  Secondary: FBX mesh + texture atlas export for Unity C# import
    - FBX version 2020.2
    - Embedded textures as PNG (not DDS)
    - Animation: baked keyframes (no constraints)
    - Scale: 1 unit = 1 cm (UE standard)
    - Up axis: Z-up
```

§5 PER-MODE PHYSICS & SCORING CONTRACTS

5.1 ArenaSettings Quick Reference

mode_id	Package Path	Slice	Balls	Target	Time	JumpScale	WalkScale	Scoring
basketball_h2h	VeniceBeach	StreetBall	1	3	∞	1.03	1.045	✅
basketball_dunk	VeniceBeach	FirstToTwentyOne	1	21	∞	1.0	1.0	✅
basketball_3v3	VeniceBeach	HalfCourtShootout	2	11	∞	1.035	1.055	✅
karate_h2h	Dojo	StreetBall	1	5	150s	—	—	✅
karate_endless	Dojo	StreetBall	1	∞	∞	—	—	✅
baseball	BaseballPark	Practice	1	6	300s	—	—	✅
football	Gridiron	StreetBall	1	3	240s	—	1.05	✅
soccer	SoccerStadium	StreetBall	1	5	180s	—	—	✅
golf	Links	Practice	1	5	300s	—	—	✅
tennis	TennisCourt	TimedBlitz	1	5	120s	—	1.04	✅
volleyball	SandCourt	TimedBlitz	1	5	120s	—	1.04	✅
surfing	VeniceBeach	Practice	0	∞	180s	1.04	1.06	✅
skateboarding	VeniceBeach*	Practice	0	∞	180s	1.06	1.05	✅
		Practice	0	∞	180s	1.05	1.04	✅

mode_id	Package Path	Slice	Balls	Target	Time	JumpScale	WalkScale	Scoring
snowboarding	Training-Floor*							
gymnastics	TrainingFloor	Practice	1	5	240s	—	—	✅
brain_brawl	NeuroArena	Practice	0	0	120s	—	—	❌
market_browse	Luma_Venice_Shop	Practice	0	0	∞	—	—	❌

`*` = incorrect venue assignment (staging bug)

5.2 Scoring Formulas (Production Modes)

```
// XP Award (all modes except brain_brawl)
xp_awarded = max(10, score / 5)
xp_cap     = 500  // MUST ADD — currently uncapped

// XP Award (brain_brawl only)
xp_awarded = score / 10

// Shard Economy (MUST ADD to backend)
shards_base = won ? 50 : (tied ? 25 : 15)
combo_bonus = combo > 3 ? combo * 5 : 0
critical_bonus = criticals * 10
shards_total = shards_base + combo_bonus + critical_bonus
pacing_bonus = PACE_EfficiencyScore >= 75 ? ceil(shards_total * 0.05) : 0
shards_awarded = shards_total + pacing_bonus

// PRQ Delta
prq_base = won ? 2.0 : (tied ? 0.5 : 0.2)
mode_weight = PRQ.modeWeight(mode)  // 0.9–1.5
combo_bonus = min(1.0, combo * 0.05)
critical_bonus = min(0.5, criticals * 0.1)
dominance_bonus = won ? min(0.5, scoreDiff * 0.05) : 0
mri_bonus = (MRI_Score / 100.0) * 0.5
prq_delta = clamp(prq_base * mode_weight + combo_bonus + critical_bonus + domin-
ance_bonus + mri_bonus, 0, 100)

// Final Score (combo-based modes)
final_score = totalStylePoints * (1.0 + PRQ_Normalized * 0.3) * (NeuralBurstActive ? 1
.2 : 1.0)
```

§6 BACKEND API DEPENDENCIES PER MODE

6.1 Universal Endpoints (All Production Modes)

Endpoint	Method	Purpose
`/api/games/session`	POST	Session receipt: mode_id, score, duration, XP, shards, PRQ delta
`/api/streaming/launch-mode`	POST	Deep link generation + live_sessions doc + sovereign broadcast
`/api/session/state`	POST	State machine: launching → map_loading → active → completed
`/ws/game/{room_id}`	WS	Real-time multiplayer score sync (realtime modes)
`/ws/sovereign`	WS	Sovereign bridge: telemetry, match_score, hardware_auth

6.2 Mode-Specific Endpoints

Mode	Endpoint	Method	Purpose
brain_brawl	`/api/brain-brawl/submit`	POST	Non-standard session (XP = score//10)
brain_brawl	`/api/brain-brawl/questions`	GET	Quiz question set
who_scene_it	`/api/games/who-scene-it`	GET	Trivia config (preview only)
court_carnival	`/api/games/court-carnival`	GET	Party board config (rename from mario-party)
court_carnival	`/api/games/court-carnival/session`	POST	Party session (rename)
market_browse	`/api/cards/purchase`	POST	Creator Card PayPal purchase

6.3 Neurocognitive Endpoints (NEW)

Endpoint	Method	Purpose
`/api/neurocognitive/session`	POST	MRI session telemetry (Tier A/B/C scores, decel curve)
`/api/neurocognitive/history`	GET	MRI score history for user (last 30 sessions)
`/api/neurocognitive/baseline`	GET	User’s ARV baseline, ESI baseline, pacing averages

---

§7 REGISTRY ALIGNMENT PATCHES

#	File	Change
1	`backend/FEL_ModeManager.production.json`	Fix total_modes 17→19; fix map paths /Maps/→/Venues/; who_scene_it→preview; mario_party_fever→court_carnival (preview)
2	`FinalEvolutionLab/Models/GameMode.swift`	Add enum cases: whoSceneIt, courtCarnival, marketBrowse + inputScheme + registry entries
3	`backend/ue_mode_maps.json`	Add: who_scene_it→Neuro_Arena, court_carnival→Venice_Beach_Court, market_browse→Sovereign_Shop
4	`infra/ue5_config/DefaultGame.ini`	Remove misrouted skateboarding/snowboarding entries; add who_scene_it + court_carnival
5	`ArenaSettings.json`	Split karate→karate_h2h+karate_endless; add who_scene_it, court_carnival; fix snowboarding venue
6	`FEL_VenueRegistry.production.json` (both)	Add karate_endless, who_scene_it, court_carnival, market_browse entries
7	`FELEmergentDeepLinkSubsystem.cpp`	Rename mario_party_fever→court_carnival in GetModeToVenueMap()
8	`backend/server.py`	Add shard+PRQ delta to create_game_session; add XP cap 500; rename mario-party→court-carnival endpoints
9	`backend/server.py`	Add neurocognitive session endpoint; MRI computation on session completion

#	File	Change
10	`SystemScanRecord.swift`	Extend UE bridge payload with neurocognitive block

---

§8 FIRST-5-VENUE PRODUCTION PRIORITY

Rank	Venue	Cooked Path	Modes	Ship
1	Venice Beach Court	`/Game/FEL/Venues/Venice-Beach/Venice-Beach`	basketball_h2h, basketball_dunk, basketball_3v3	v1.0
2	Zen Dojo	`/Game/FEL/Venues/Dojo/Dojo`	karate_h2h, karate_endless	v1.0
3	Baseball Park	`/Game/FEL/Venues/Baseball-Park/Baseball-Park`	baseball	v1.0
4	Gridiron Stadium	`/Game/FEL/Venues/Gridiron/Gridiron`	football	v1.0
5	Soccer Stadium	`/Game/FEL/Venues/SoccerStadium/SoccerStadium`	soccer	v1.0

8 modes across 5 venues for v1.0 release.

---

§9 DO NOT SHIP UNTIL — CRITICAL GATES

#	Gate	Status	Blocks
G1	`fel_prebuild_ci_check.sh --strict` passes	⚠️	All builds
G2	All 12 production deep links resolve correctly	⚠️	Ship
G3	ModeManager total_modes matches entry count	❌ 17≠19	All builds
G4	who_scene_it + court_carnival NOT “production”	❌	Ship
G5	EmergentPlayMap no misrouted staging entries	❌	Ship
G6	.app bundle has cookeddata/.pak	Verify	iOS
G7	CFBundleIdentifier correct	✅	iOS
G8	URL scheme re-gistered	✅	iOS
G9	All 12 modes post valid session receipts	✅	Ship
G10	No AltStore/ SideStore/OTA refer-ences	✅	App Store
G11	Backend awards shards on session complete	❌	Economy
G12	Backend computes PRQ delta on session complete	❌	Economy
G13	Server-side XP cap ≤500/session	❌	Exploit prevention

§10 DESIGN SYSTEM

Property	Value
Theme	Dark Clinical
Background	#050505 (default), #0F0F13 (paper), #16161A (card)
Primary	#00E5FF (cyan)
Accent Alert	#FF3366
Accent Success	#00FF9D
Neuro Accent	#A855F7 (purple)
Heading Font	Barlow Condensed
Body Font	IBM Plex Sans
Mono Font	JetBrains Mono

Generated from `anti-gravity-fel` branch — Commit: 9519541
Date: 2026-05-22
Classification: Seele AI Compilation Target — Production Specification v1.0